Creating CI/CD pipelines in Microsoft Azure involves using Azure DevOps Services, which provides a comprehensive suite of tools for managing the entire lifecycle of your application development. Here's a step-by-step guide to create a CI/CD pipeline in Azure:

Step 1: Set Up Your Azure DevOps Environment

1. **Create an Azure DevOps Organization**: If you don't have one, [create an Azure DevOps account](https://dev.azure.com/).
2. **Create a Project**:
   - Go to your Azure DevOps organization.
   - Click "New Project".
   - Enter a name for your project and click "Create".

Step 2: Create a Repository

1. **Navigate to Repos**:
   - Click on "Repos" in your Azure DevOps project.
   - Click "Files" and then "Initialize" to create a new repository.
   - You can also import a repository from GitHub or another source.

Step 3: Create a Build Pipeline

1. **Go to Pipelines**:
   - Click on "Pipelines" and then "Create Pipeline".
   - Select your repository source (Azure Repos Git, GitHub, etc.).

2. **Configure the Pipeline**:
   - Choose a YAML pipeline or start with the classic editor.
   - For YAML, Azure DevOps provides templates to get you started. You can also define your pipeline configuration in a `azure-pipelines.yml` file.

3. **Define the Pipeline Configuration (YAML)**:

- Here’s an example `azure-pipelines.yml` for a Node.js application:

```yaml
trigger:
- main

pool:
  vmImage: 'ubuntu-latest'

steps:
- task: UseNode@2
  inputs:
    version: '12.x'
- script: |
    npm install
    npm run build
    npm test
  displayName: 'Install, build, and test'
- task: PublishBuildArtifacts@1
  inputs:
    pathToPublish: '$(Build.ArtifactStagingDirectory)'
    artifactName: 'drop'
    publishLocation: 'Container'
```

4. **Save and Run the Pipeline**:
   - Save the pipeline configuration.
   - Run the pipeline to ensure everything is set up correctly.

Step 4: Create a Release Pipeline

1. **Go to Releases**:
   - Click on "Releases" under Pipelines.
   - Click "New pipeline" and select a template or start with an empty job.

2. **Configure the Release Pipeline**:
   - Add an artifact by selecting the build pipeline created in the previous step.
   - Define the stages (e.g., Development, Staging, Production).

3. **Add Tasks to Stages**:
   - Add tasks to each stage to deploy your application.
   - For example, to deploy to Azure App Service:
     - Add an "Azure App Service Deploy" task.
     - Configure the task with your Azure subscription, App Service name, and package/file path.

4. **Define Continuous Deployment Triggers**:
   - Configure the pipeline to trigger deployments automatically after a build completes.

5. **Save and Create the Release**:
   - Save the release pipeline configuration.
   - Create a release to deploy your application.

Example CI/CD Workflow

Here's a summary of the steps involved in a CI/CD workflow:

1. **Code Commit**:
   - Developers commit code to the repository.

2. **Continuous Integration**:
   - The build pipeline triggers automatically on code commit.

- The pipeline installs dependencies, builds the application, runs tests, and publishes build artifacts.

3. **Continuous Deployment**:

- The release pipeline triggers automatically after a successful build.

- The pipeline deploys the application to the specified environments (e.g., Development, Staging, Production).

Additional Tips

- **Secrets Management**: Use Azure Key Vault to securely manage secrets and credentials.

- **Environment Variables**: Use pipeline variables to manage environment-specific configurations.

- **Approvals and Gates**: Configure manual approvals and automated gates for your release pipeline to ensure quality control.

- **Monitoring and Alerts**: Set up monitoring and alerting to stay informed about the health and performance of your pipeline and deployed applications.

By following these steps, you can set up a comprehensive CI/CD pipeline in Azure, leveraging Azure DevOps Services to automate your build, test, and deployment processes.